Learned congestion control

Mar 12 2026

Need for congestion control

- Applications not aware of available network bandwidth
- Shared bottleneck links -> competing flows must coordinate
- Under sending -> under utilization
- Over sending -> long delays, loss

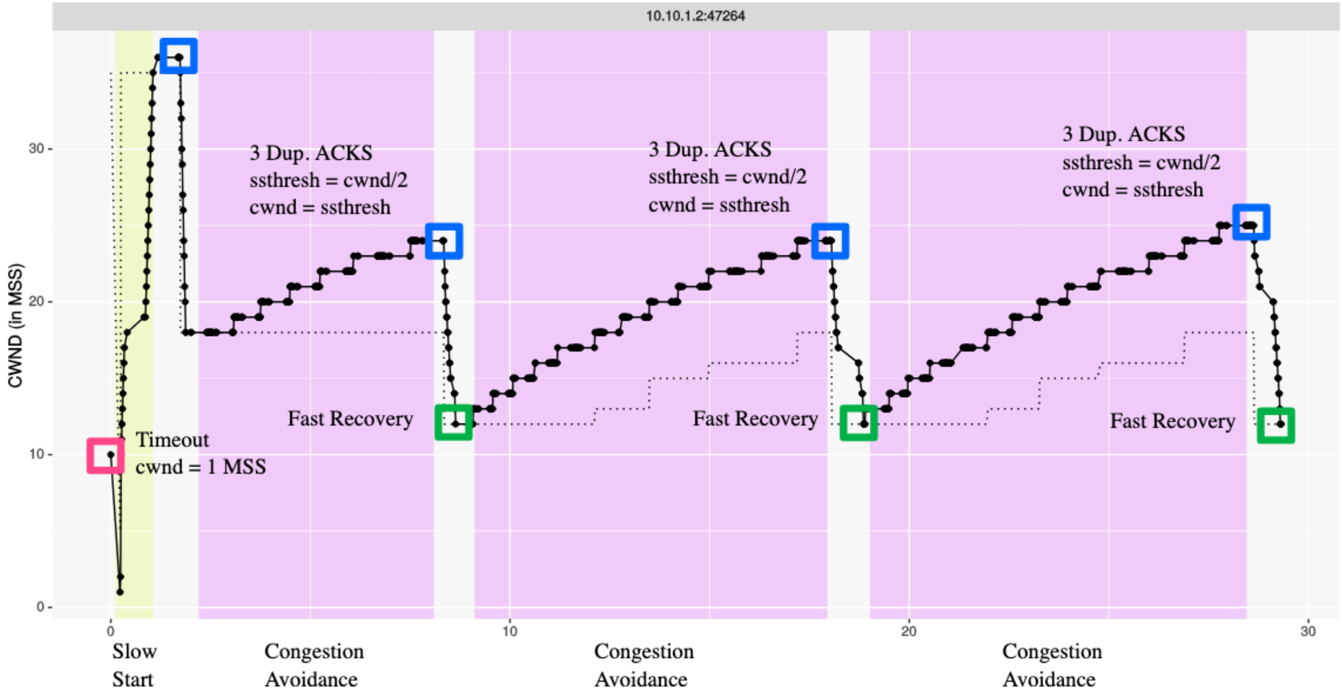
- Challenge: No single “ground truth” signal

What is CCA actually solving?

- Sender does not know
  - Available bandwidth / bottleneck
  - Number/behavior of competing flows
- Must infer congestion from indirect signals
  - Packet loss, delay, ACK arrival patterns
- Sender controls the sending rate and congestion window

Congestion control approaches

- End-end control (no network feedback)
  - Senders try to estimate network state
  - Example: TCP AIMD, CUBIC



Congestion control approaches

- End-end control (no network feedback)
  - Senders try to estimate network state
  - Example: TCP AIMD, CUBIC
- Network-assisted control
  - Example: ECN (Explicit congestion notification)
- Delay-based approaches
  - Use RTT to estimate congestion
  - Does not force/induce loss
  - Example: BBR (Bottleneck Bandwidth and Round-trip)

Why still unsolved?

- Diverse and dynamic network conditions
- Inter-workload dynamics
- Balancing fairness and performance, throughput vs latency, responsiveness vs stability
- Partial observability
  - Signals are ambigous